

Azure SQL Database per sviluppatori: tutte le novità da non perdere

Gianluca Hotz
Presidente UGISS.ORG
ghotz@ugiss.org | @glhotz



About me

- Gianluca Hotz | @glhotz | ghotz@ugiss.org
- Consulente Indipendente
 - 30 anni con SQL Server (dalla versione 4.21 nel 1996)
 - «Database modeling & development, sizing & administration, modernization (upgrades & migrations), performance tuning, security»
- YNAP SQL Server DBA Team (part-time)
- «Community»
 - 25+ anni Microsoft MVP SQL Server/Data Platform (dal 1998)
 - «VMware Experts SQL Server»
 - Fondatore e presidente UGISS (User Group Italiano SQL Server)



Introduzione

- SQL Server 2025 ~ Azure SQL Database e Managed Instance
- Feature in preview in SQL Server 2025/Azure SQL Database
 - ALTER DATABASE SCOPED CONFIGURATION SET
PREVIEW_FEATURES = ON;
- GA/Preview in Azure SQL Managed Instance
 - <https://learn.microsoft.com/en-us/azure/azure-sql/managed-instance/update-policy>

Supporto JSON prima di 2025

- FOR JSON e OPENJSON (dalla 2016)
 - FOR JSON PATH / AUTO per serializzare query come JSON
 - OPENJSON per leggere dati JSON come tabella relazionale
- Funzioni scalari JSON
 - ISJSON, JSON_VALUE, JSON_QUERY, JSON_MODIFY
 - Lettura e aggiornamento di singoli valori tramite path expression
- Colonne calcolate e indici su JSON (dal 2017)
 - Indici su colonne calcolate con JSON_VALUE per accelerare le query
- Limiti principali
 - Nessun tipo nativo JSON: i dati vengono archiviati come NVARCHAR
 - Assenza di JSON_ARRAY, JSON_OBJECT e JSON_ARRAYAGG
 - Validazione e vincoli di schema non disponibili nativamente

Novità JSON in SQL Server 2025

- Tipo nativo JSON
 - Archiviazione binaria ottimizzata: niente più NVARCHAR(MAX)
 - Validazione automatica del documento al momento dell'inserimento
 - Supporto documenti fino a 2 GB
- Indice JSON nativo (CREATE JSON INDEX)
 - Indicizzazione diretta delle proprietà JSON senza colonne calcolate
 - Un indice per colonna con più path specificabili
- Nuove funzioni
 - JSON_ARRAYAGG e JSON_OBJECTAGG per aggregazioni native
 - JSON_CONTAINS per ricerche di contenimento su documenti JSON
 - Metodo .modify() per aggiornamenti in-place sulla colonna JSON

Demo

JSON



Indici JSON e limitazioni

- Solo colonne di tipo JSON nativo
 - No VARCHAR(MAX)/ NVARCHAR(MAX)
- Un solo JSON index per colonna
 - si possono includere piu path nello stesso indice
- Clustered primary key obbligatoria
- Non applicabile a
 - viste, variabili tabella, tabelle memory-optimized, colonne JSON calcolate
- Build online non supportata
- Compressione e MAXDOP ignorati
- Query hint per uso indice non supportato

Disponibilità funzionalità JSON

- Azure SQL Database e SQL Managed Instance (update policy)
 - Data type nativo e JSON_OBJECTAGG(), JSON_ARRAYAGG()
 - No indici JSON
 - No funzioni JSON_CONTAINS() e .modify()
 - No Array wildcard nel path (es. \$[*], range, last)
- SQL Server 2025: Preview ma senza database scoped config

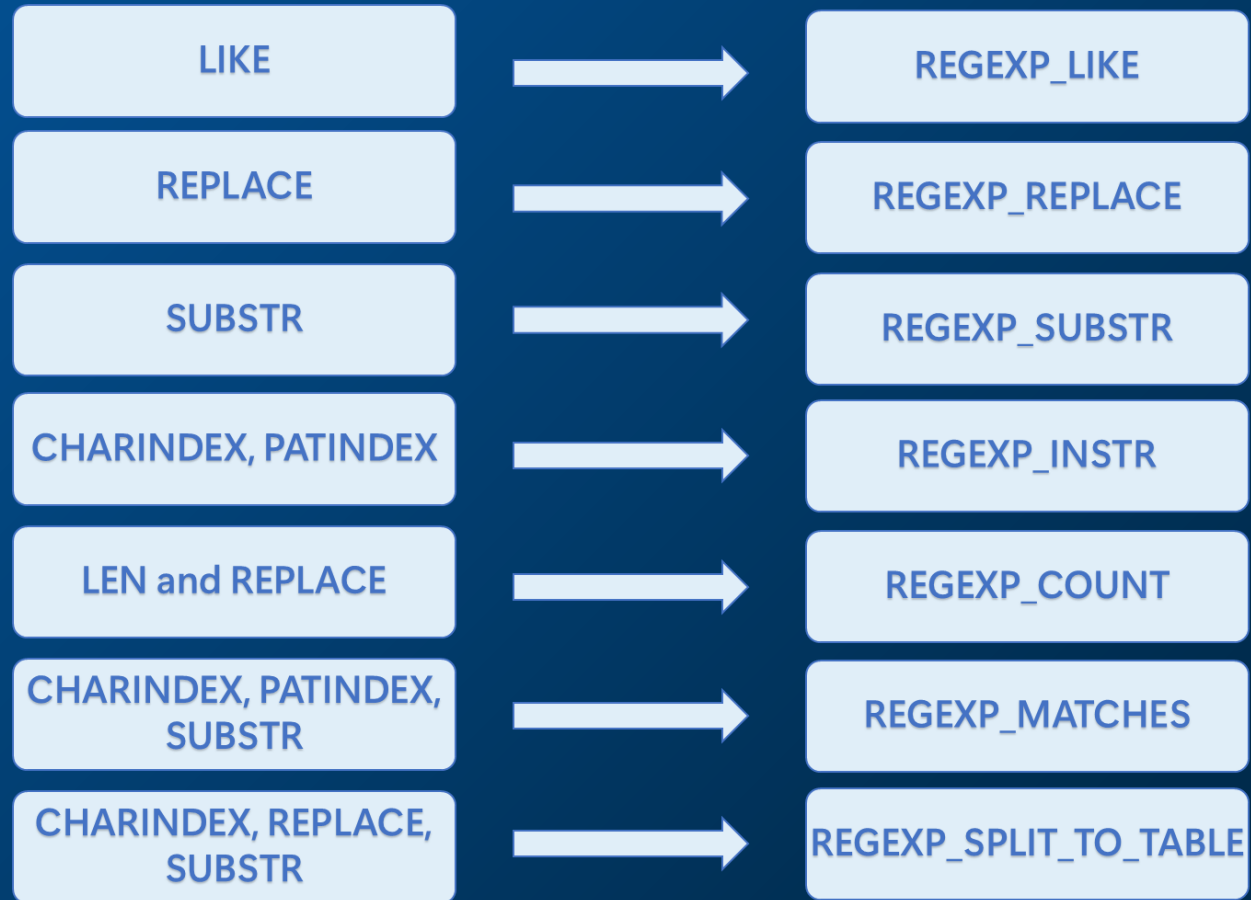
Regex prima di SQL Server 2025

- LIKE e PATINDEX
 - pattern matching basilare con wildcard (% , _ , [])
- Combinazione CHARINDEX / SUBSTRING / REPLACE
 - logica T-SQL verbosa e fragile per estrarre o sostituire pattern semplici
- CLR Integration (.NET)
 - funzioni C# con System.Text.RegularExpressions caricate in SQL Server
 - potenti ma richiedono permessi UNSAFE e overhead di context switching
- SQL Server Extensibility Framework
 - Scripting R, Python, Java, .NET
 - Eccessivo solo per Regex...

Regex in SQL Server 2025

Funzione	Tipo	Descrizione
REGEXP_LIKE	Scalare	Restituisce TRUE se la stringa di input corrisponde al pattern Regex
REGEXP_COUNT	Scalare	Conta il numero di volte che un pattern ricorre in una stringa
REGEXP_INSTR	Scalare	Restituisce la posizione della corrispondenza di un pattern in una stringa
REGEXP_REPLACE	Scalare	Sostituisce le sottostringhe che corrispondono a un pattern con una stringa di sostituzione
REGEXP_SUBSTR	Scalare	Estrae una sottostringa che corrisponde a un pattern
REGEXP_MATCHES	TVF	Restituisce una tabella con tutte le corrispondenze, incluse le sottostringhe e le loro posizioni
REGEXP_SPLIT_TO_TABLE	TVF	Suddivide una stringa in righe usando un delimitatore Regex

Mapping T-SQL tradizionale e Regex



Demo

Regex



Note Regex in SQL Server 2025

- Implementazione basata su motore RE2 di Google
 - garanzia di esecuzione lineare
 - nessun rischio di ReDoS (catastrophic backtracking) ma lookahead e lookbehind non supportati
- Compatibility level 170 richiesto
 - per REGEXP_LIKE, REGEXP_MATCHES e REGEXP_SPLIT_TO_TABLE
 - le altre quattro funzioni sono disponibili a tutti i livelli di compatibilità
- Limiti
 - Non usa indici (Non SARGable)
 - REGEXP_LIKE in WHERE provoca una full scan
 - abbinare un filtro LIKE se possibile per restringere il dataset prima dell'applicazione della regex
 - Non supportata con tabelle memory-optimized
 - Dimensione massima LOB di 2 MB

Risorse Regex

- Annuncio GA
 - <https://devblogs.microsoft.com/azure-sql/general-availability-announcement-regex-support-in-sql-server-2025-azure-sql>
- Presentazione a Data Saturday Pordenone 2026
 - Erland Sommarskog (una garanzia!)
 - Demo con esempi passo passo anche di casi particolari
 - [https://www.sommarskog.se/present/RegExp%20\(DataSat%2080\).zip](https://www.sommarskog.se/present/RegExp%20(DataSat%2080).zip)

Nuove funzioni T-SQL

- PRODUCT()
- BASE64_ENCODE()
 - RFC 4648 Table 1 e Table 2 con parametro "URL safe"
- BASE64_DECODE()
 - Ignora whitespace
- CURRENT_DATE()
- UNISTR()
- Concatenazione stringhe ANSI ||
- Assegnazione stringhe composta ||=

T-SQL Fuzzy String Match (Preview)

- Legacy
 - SOUNDEX() e DIFFERENCE()
- Nuove basate su algortimo Damerau-Levenshtein
 - EDIT_DISTANCE()
 - EDIT_DISTANCE_SIMILARITY()
- Nuove basate su algortimo Jaro-Winkler
 - JARO_WINKLER_DISTANCE()
 - JARO_WINKLER_SIMILARITY()

Demo

T-SQL



Chiamate a REST API

- EXEC sp_invoke_external_rest_endpoint
- Limiti di dimensione
 - payload massimo 100 MB (UTF-8)
 - URL massimo 8 KB; query string massima 4 KB
 - header totali massimo 8 KB
- Retry automatico configurabile
- Non richiamabile da funzioni T-SQL (UDF)
 - utilizzabile solo da stored procedure
 - per batch di righe -> aggregare dati in un unico JSON

Sicurezza chiamate a REST API

- SQL Server 2025 e Azure SQL Managed Instance
 - EXECUTE sp_configure 'external rest endpoint enabled', 1;
- GRANT EXECUTE ANY EXTERNAL ENDPOINT
 - Solo endpoint HTTPS con TLS
- Per Azure SQL Database e SQL Managed Instance
 - Whitelist endpoint autorizzati
 - Altri endpoint vanno esposti tramite API Management
- Autenticazione tramite database-scoped credential
 - HTTPEndpointHeaders, HTTPEndpointQueryString, Managed Identity e Shared Access Signature

Demo

REST API



«Optimized Locking»

- Grandi transazioni
 - Riduce numero di lock usati (di conseguenza la memoria)
 - Aumenta concorrenza evitando «lock escalation»
- «Transaction ID (TID) locking»
 - Richiede attivazione «Accelerated Database Recovery» a livello di database
 - ALTER DATABASE ... SET OPTIMIZED_LOCKING = ON
 - Default non attivo su SQL Server 2025, attivo su Azure SQL Database e MI
- «Lock After Qualification (LAQ)»
 - Richiede attivazione «Read Committed Snapshot Isolation»
 - e «Transaction ID (TID) locking»

Demo

Optimized Locking



Operazioni DML senza «TID Locking»

- Qualificano righe prima di modificarle
 - Acquisizione di «U-Lock» per la qualifica delle righe
 - conversione in «X-Lock» per righe qualificate, mantenuti fino a fine transazione
- Situazioni di blocco comuni
 - Di acquisizione (WAIT) «U-Lock»
 - qualcuno sta mantenendo un «U-Lock» acquisito per qualifica
 - qualcuno sta mantenendo un «X-Lock» acquisito per modifica
 - Di conversione (CNVT) da «U-Lock» a «X-Lock»
 - qualcuno mantenendo un «S-Lock» acquisito per lettura

«Transaction ID (TID) locking»

- Con «ADR» ogni riga marchiata con TID transazione che l'ha modificata
- Lock di tipo «XACT» acquisti su TID di modifica non su chiavi (o RID) righe
 - transazione che aggiorna mantiene solo «X-lock» su TID
 - altre transazioni aspettano con «S-lock» su TID + riga per leggere/modificare
- Qualifica delle righe avviene senza «U-Lock»
 - Lettura per qualifica eventualmente si blocca con «S-lock» su TID + riga
 - Altrimenti modifica immediata acquisendo comunque «X-lock» su pagina/riga
 - solo per durata della modifica, rilasciati immediatamente
 - rimane «X-Lock» su TID fino a fine transazione
 - evita «lock escalation» a livello di tabella e risparmia memoria lock
 - Rimane fondamentalmente approccio pessimistico («writer-block-reader»)

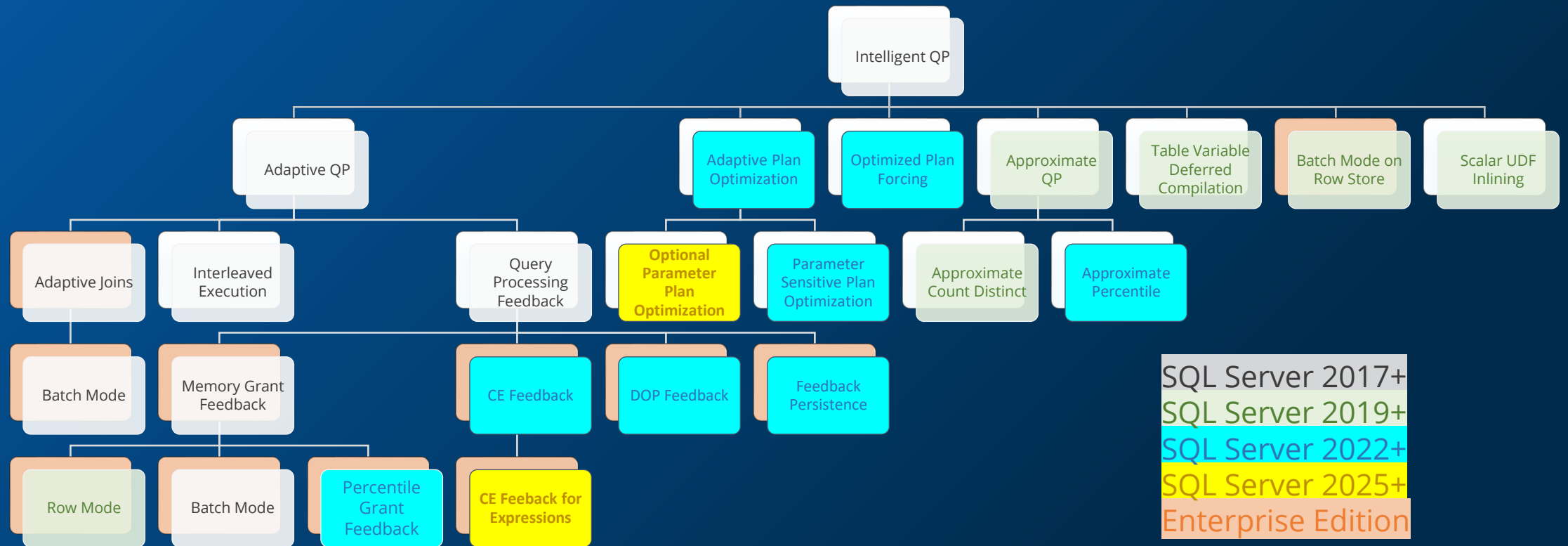
«Lock After Qualification (LAQ)»

- RCSI approccio ottimistico: valutazione del predicato senza lock
- Qualifica delle righe con lettura ultimo valore confermato
 - Riga non qualificata passa alla prossima (nessun lock richiesto)
 - Riga qualificata viene aggiornata
 - se altra transazione ha «X-Lock» su TID riga, aspetta con logica «S-Lock» TID + riga
 - quando lock rilasciato, riga viene riqualificata perché predicato potrebbe non valere più
 - se ancora qualificata, viene modificata altrimenti si passa alla prossima
 - modifica acquisendo comunque «X-lock» su pagina/riga
 - solo per durata della modifica, rilasciati immediatamente
 - rimane «X-Lock» su TID fino a fine transazione
- Meccanismo di «feedback» riqualifica troppo onerosa
 - basato su piani esecuzione (cache/Query Store)
 - Abilita/disabilita LAQ

Articoli «Optimized Locking»

- Serie di Sergio Govoni in italiano
 - SQL Server 2025 Optimized Locking
 - <https://www.ugiss.org/2025/07/07/sql-server-2025-optimized-locking>
 - SQL Server 2025 Optimized Locking Internals: Gestione dei lock in aggiornamenti concorrenti
 - <https://www.ugiss.org/2025/10/29/sql-server-2025-optimized-locking-internals-gestione-dei-lock-in-aggiornamenti-concorrenti>
 - SQL Server 2025 Optimized Locking: TID internals
 - <https://www.ugiss.org/2025/11/14/sql-server-2025-optimized-locking-tid-internals>
- Introducing optimized locking v2
 - <https://techcommunity.microsoft.com/blog/sqlserver/introducing-optimized-locking-v2/4468745>

Intelligent Query Processing Next Gen



«CE Feedback for Expressions»

- Evoluzione funzionalità «CE Feedback»
 - Analizza query con stime errate di intere query e valuta diversi modelli
 - Se modello diverso più efficace, aggiunge «Query Store Hint»
 - Correzione vale solo per quella query
- Estensione alle espressioni
 - Analizza stime errate per singole espressioni (filtri/join)
 - Correzione applicata a più query
- «Database Scoped Configuration»
 - A partire da livello di compatibilità 160 (ma in 2022 non c'è)

«Optional parameter plan optimization (OPPO)»

- Evoluzione funzionalità «Parameter Sensitive Plan(PSP)»
 - Problema «parameter sniffing» con distribuzione sbilanciata valori parametri
 - Piano «dispatcher» per varianti di piano
- Estensione a «parameter sniffing» con parametri opzionali
 - Pattern tipico WHERE col = @p OR @p IS NULL
 - Es. genera un piano con «Seek» e uno con «Scan»
- «Database Scoped Configuration»
 - A partire da livello di compatibilità 170

Varie IQP SQL Server 2025

- Funzionalità attive di default in IQP SQL Server 2025
 - «Degree of parallelism (DOP) feedback»
 - «Query Store» per repliche secondarie in sola lettura
- «ABORT_QUERY_EXECUTION Query Hint»
 - Previene esecuzione query es. applicazione non modificabile in breve tempo
 - Da usare come «Query Store Hint»

Demo

«ABORT_QUERY_EXECUTION Query Hint»



«Optimized sp_execute_sql»

- Previene «compilation storm»
 - elevato numero di query compilato simultaneamente
 - Invocazioni identiche stesso batch compilate in parallelo salvano più copie del piano
- Database Scoped Configuration
 - Serializza tramite lock di compilazione batch identici al netto dei parametri
 - Prima compilazione va in cache, altre la riutilizzano

Tempdb

- Controllo spazio tempdb con «Resource Governor»
 - GROUP_MAX_TEMPDB_DATA_MB
 - GROUP_MAX_TEMPDB_DATA_PERCENT
 - <https://learn.microsoft.com/en-us/sql/relational-databases/resource-governor/tempdb-space-resource-governance>
- «Accelerated Database Recovery» per tempdb
 - Troncamento T-Log più aggressivo e «rollback istantaneo», no «Recovery» più veloce
 - <https://learn.microsoft.com/en-us/sql/relational-databases/accelerated-database-recovery-concepts?#adr-improvements-in-sql-server-2025>
- Supporto per tempdb su tmpfs versione Linux
 - «tempdb in RAM»
 - <https://learn.microsoft.com/en-us/sql/linux/sql-server-linux-tmpfs-tempdb>

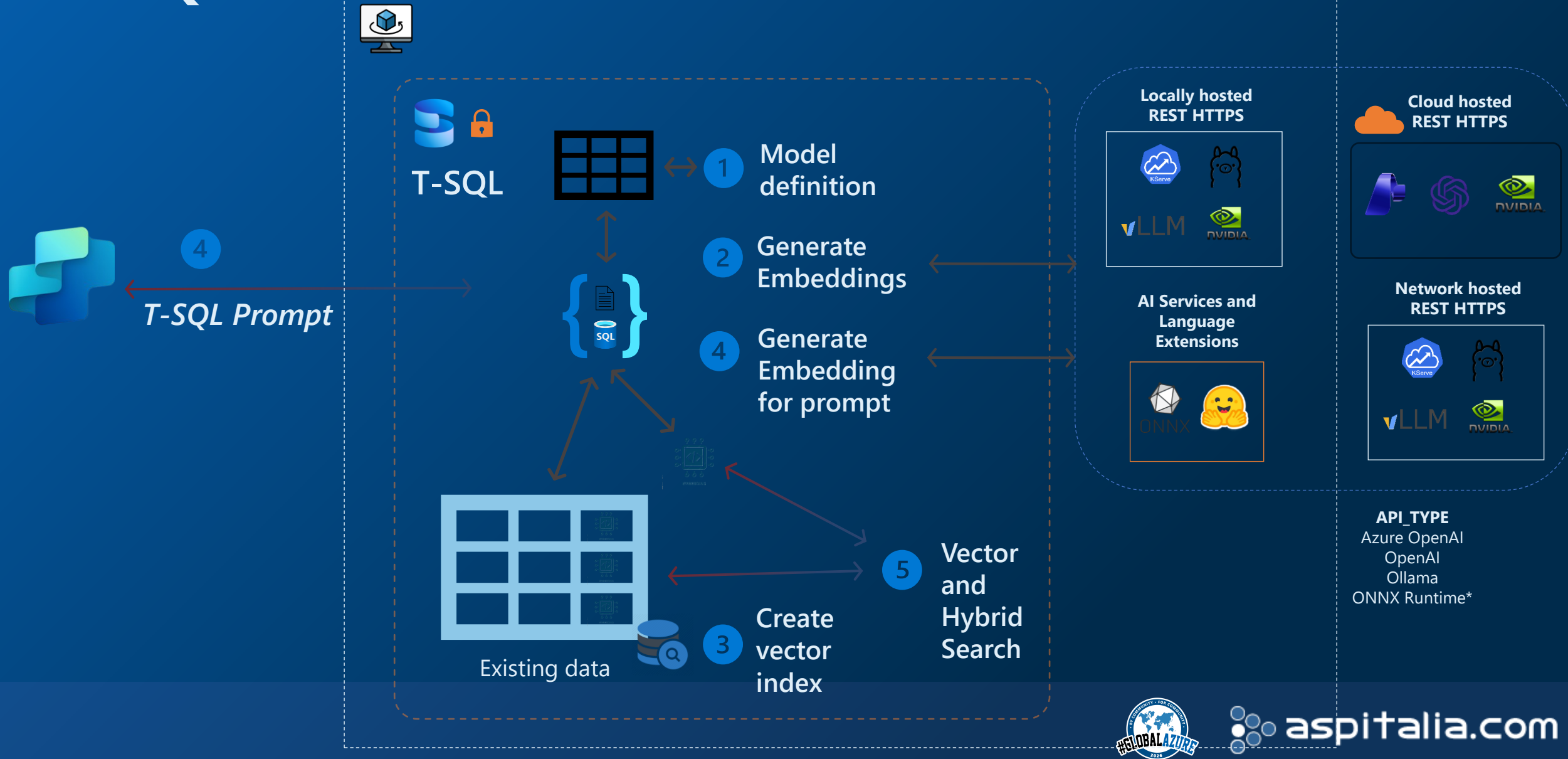
«Columstore»

- Indici «non-clustered» posso essere ordinati
- Indici ordinati possono essere creati/ricreati ONLINE
 - Utilizza tempdb e deve essere specificato MAXDOP = 1
- Operazioni «shrink» migliorate
 - Pagine LOB possono essere spostate
 - Es. varchar(max), nvarchar(max), varbinary(max)

Altri miglioramenti

- «Adaptive shallow cleanup» migliora «autocleanup» di «Change Tracking»
 - <https://learn.microsoft.com/en-us/sql/relational-databases/track-changes/about-change-tracking-sql-server?#sql-server-2025-changes>
- Statistiche persistenti per repliche secondarie in sola lettura
 - In precedenza usavano tempdb ora Query Store
 - <https://learn.microsoft.com/en-us/sql/relational-databases/performance/persisted-stats-secondary-replicas>

SQL Server 2025 Vector Architecture



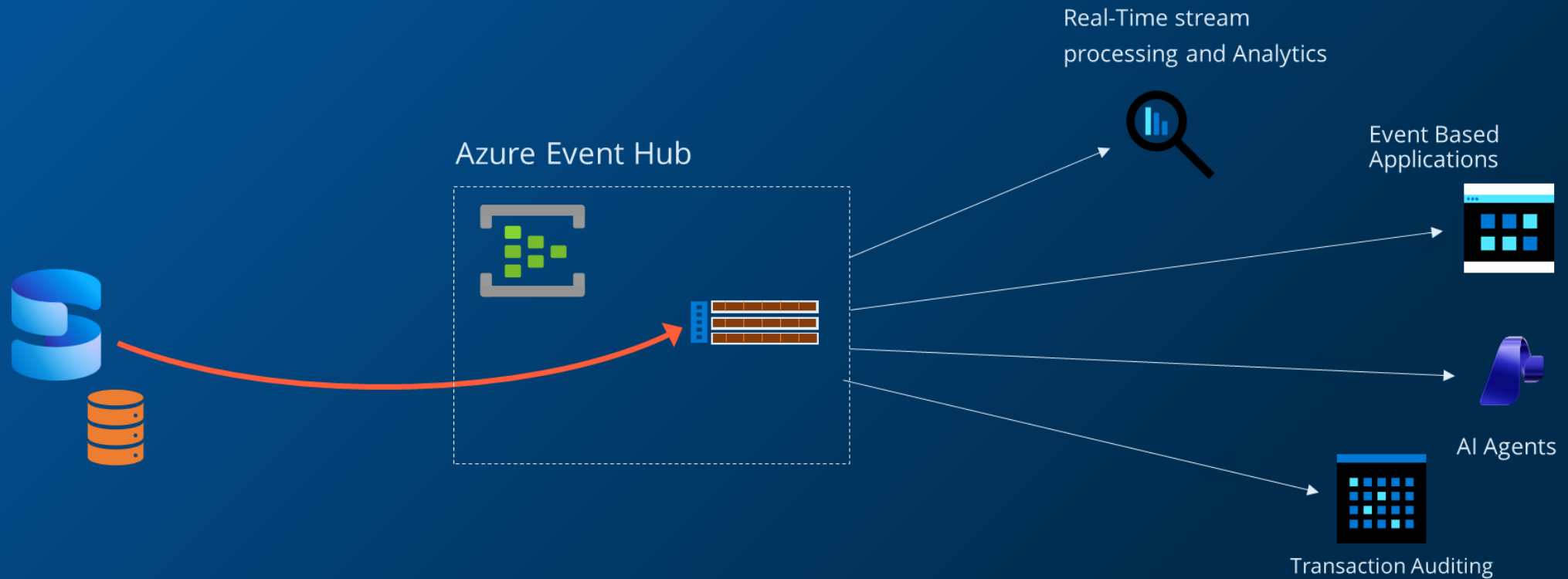
Vettori ed embeddings nativi

Componente	Descrizione
VECTOR(n)	Tipo nativo per archiviare vettori ad alta dimensionalita in colonne SQL; supporta fino a circa 4.000 dimensioni in half-precision (16-bit)
VECTOR_DISTANCE	Calcola la distanza tra due vettori con metriche cosine (testo/NLP), euclidean (numerico) e dot product (vettori normalizzati)
AI_GENERATE_EMBEDDINGS	Genera embeddings da testo direttamente in T-SQL invocando un modello esterno registrato (Azure OpenAI, Ollama, ecc.)
EXTERNAL MODEL	Registra modelli AI remoti come oggetti di primo livello nel database; supporta Azure OpenAI, endpoint OpenAI-compatibili e modelli locali via Ollama
VECTOR_SEARCH	Ricerca approximate nearest neighbor (ANN) tramite indice DiskANN; ottimizzata per RAG, semantic search e sistemi di raccomandazione (preview)
CREATE VECTOR INDEX	Crea un indice DiskANN sulla colonna VECTOR per ricerche veloci e scalabili; abilitato con PREVIEW_FEATURES = ON

Integrazione MCP

Componente	Descrizione
SQL MCP Server (DAB)	Server open source Microsoft basato su Data API builder; espone operazioni SQL come tool MCP per agenti AI via HTTP streamable o stdio
Protocollo MCP	Standard aperto (v2025-06-18) che definisce come gli agenti AI scoprono e invocano tool esterni in modo predicibile e deterministico
NL2DAB (no NL2SQL)	L'agente usa entità DAB predefinite invece di generare SQL libero: approccio deterministico, senza rischi di query errate o non previste
Tool DML esposti	7 tool CRUD: read, create, update, delete, aggregate e esecuzione stored procedure; focalizzati su dati, non su schema (solo DML, no DDL)
RBAC integrato	Stessi ruoli di Data API builder applicati automaticamente a ogni tool MCP: nessuna configurazione aggiuntiva rispetto a REST o GraphQL
Caching e telemetria	Caching L1/L2 per ridurre il carico sul database; log e telemetria centralizzati per monitorare ogni interazione dell'agente AI

Change Event Streaming (Preview)



Capturing changes in SQL Server

	Change Tracking (CT)	Change Data Capture (CDC)	Change Event Streaming (CES)
Capture Source	As part of committed transactions (sync)	Committed transactions from transaction log (async). Requires SQL Server Agent	Committed transactions from transaction log (async)
Capture Destination	System table (minimum)	System table (extra logging required)	Event stream ("IOless")
Data Captured	Row identifier (DDL allowed)	Row identifier and data	Row identifier and data (DDL allowed)
Capture Method	Query system tables and user tables. Multiple destinations	Query system tables. Multiple destinations	Process event stream ("IOless"). Single destination
Availability	Immediate (pull-based)	Near real-time (pull-based)	Near real-time (push-based)
SQL effects	Transaction commit	Transaction log truncation	Transaction log truncation
Typical Use Cases	Cache invalidation and sync	Data warehousing and auditing	Event-driven architectures – microservices integration, real-time analytics, cache sync, AI Agents

@glhotz
ghotz@ugiss.org

Slide e materiale su
<https://aspit.co/globalazure-26>

 **aspitalia.com**

Lodestar
IT'S TIME TO INNOVATE

